

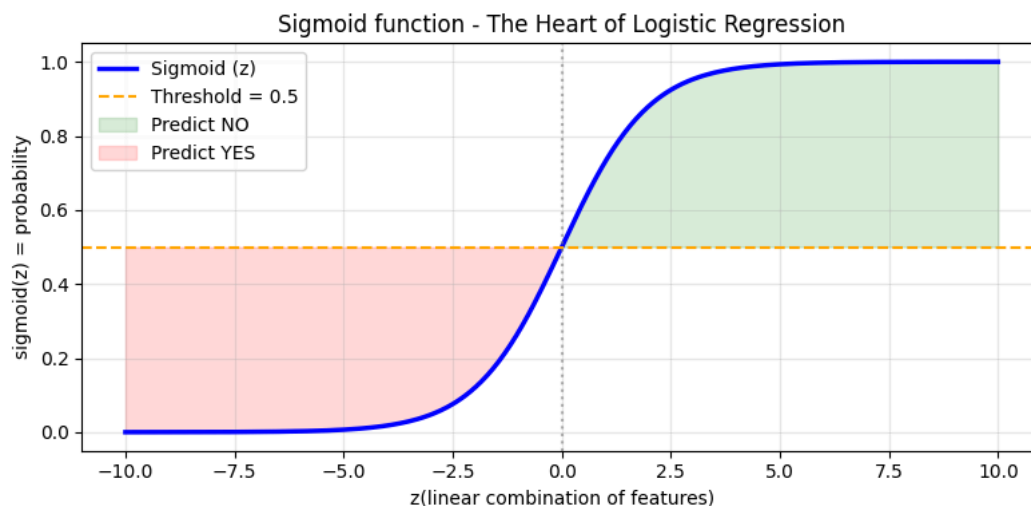
Day 9

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
X=np.array([[1],[2],[3],[4],[5],[6],[7],[8]])
y_reg=np.array([30,40,45,55,65,70,80,90])
y_cls=np.array([0,0,0,0,1,1,1,1])
lin=LinearRegression()
lin.fit(X,y_reg)
print('Regression output:', lin.predict([[5.5]]))
log = LogisticRegression()
log.fit(X,y_cls)
print('Classification output:', log.predict([[5.5]]))
print('Probability output:', log.predict_proba([[5.5]]))
print('Probability output:', log.predict_proba([[4.7]]))
print('Probability output:', log.predict_proba([[4.6]]))
print('Probability output:', log.predict_proba([[4.5]]))

print('Regression output:', lin.predict([[0.73]]))
print('Classification output:', log.predict([[0.73]]))
print('Probability output:', log.predict_proba([[0.73]]))
```

```
Regression output: [67.76785714]
Classification output: [1]
Probability output: [[0.2368932 0.7631068]]
Probability output: [[0.44177803 0.55822197]]
Probability output: [[0.470791 0.529209]]
Probability output: [[0.50000269 0.49999731]]
Regression output: [27.73392857]
Classification output: [0]
Probability output: [[0.98799263 0.01200737]]
```

```
import numpy as np
import matplotlib.pyplot as plt
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
z = np.linspace(-10, 10, 300)
prob = sigmoid(z)
plt.figure(figsize=(8,4))
plt.plot(z, prob, color='blue', linewidth=2.5, label='Sigmoid (z)')
plt.axhline(y=0.5, color='orange', linestyle='--', label='Threshold = 0.5')
plt.axvline(x=0, color='gray', linestyle=':', alpha=0.6)
plt.fill_between(z, prob, 0.5, where=(prob>0.5),alpha=0.15,color='green ',label='Predict NO')
plt.fill_between(z, prob, 0.5, where=(prob<0.5),alpha=0.15,color='red',label='Predict YES')
plt.xlabel('z(linear combination of features)')
plt.ylabel('sigmoid(z) = probability')
plt.title('Sigmoid function - The Heart of Logistic Regression')
plt.legend();plt.grid(alpha=0.3); plt.tight_layout();
plt.show()
print(f'sigmoid(0) = {sigmoid(0):.4f}')
print(f'sigmoid(5) = {sigmoid(5):.4f}')
print(f'sigmoid(-5) = {sigmoid(-5):.4f}')
```



```
sigmoid(0) = 0.5000
sigmoid(5) = 0.9933
sigmoid(-5) = 0.0067
```

```

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import precision_score, recall_score, f1_score
from sklearn.model_selection import train_test_split
X, y = make_classification(n_samples=500, n_features=5, weights=[0.7, 0.3], random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LogisticRegression();
model.fit(X_train, y_train)
probs = model.predict_proba(X_test)[: , 1]
print(f'{'Threshold':>10} {'Precision':>10} {'Recall':>8} {'F1':>6}')
print('-'* 45)
for Threshold in [0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    preds = (probs >= Threshold).astype(int)
    p = precision_score(y_test, preds, zero_division=0)
    r = recall_score(y_test, preds)
    f = f1_score(y_test, preds, zero_division=0)
    print(f'{'Threshold' :>10.1f} {'p:>10.3f} {'r:>8.3f} {'f:>6.3f}')
```

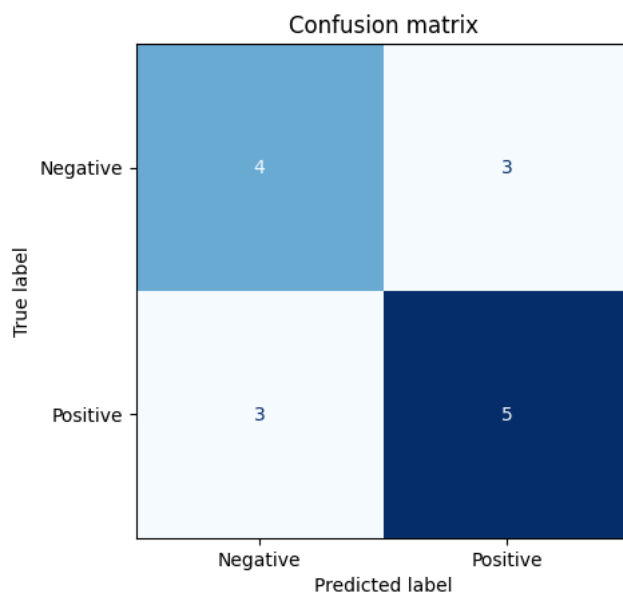
Threshold	Precision	Recall	F1
0.2	0.696	0.941	0.800
0.3	0.762	0.941	0.842
0.4	0.882	0.882	0.882
0.5	0.882	0.882	0.882
0.6	0.903	0.824	0.862
0.7	0.903	0.824	0.862
0.8	0.958	0.676	0.793

```

import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
y_true = [1,1,1,1,1, 0,0,0,0,0, 1,0,1,0,1]
y_pred = [1,1,1,0,0, 0,0,0,1,1, 1,0,0,1,1]
y_true = np.array(y_true)
y_pred = np.array(y_pred)
TP = ((y_pred == 1) & (y_true == 1)).sum() # 1,1 = tp
TN = ((y_pred == 0) & (y_true == 0)).sum() # 0,0 = tn
FP = ((y_pred == 1) & (y_true == 0)).sum() # 1,0 = fp
FN = ((y_pred == 0) & (y_true == 1)).sum() # 0,1 = fn
#print(f'TP = {TP}, TN = {TN}, FP = {FP}, FN = {FN}')
#print(f'Accuracy = {(TP + TN) / Total = ({TP} + {TN})/{len(y_true)} = {(TP+TN)/LEN(y_true):.2%}')
#print(f'Precision = TP/(TP+FP)')
#METHOD 2
cm = confusion_matrix(y_true, y_pred)
print('Confusion matrix:\n', cm)

disp = ConfusionMatrixDisplay(cm, display_labels=['Negative', 'Positive'])
disp.plot(cmap='Blues', colorbar=False)
plt.title('Confusion matrix');
plt.show()
```

Confusion matrix:
[[4 3]
[3 5]]



```

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score, accuracy_score
```

```

from sklearn.model_selection import train_test_split
import numpy as np
X_bal,y_bal = make_classification(n_samples=1000, weights=[0.5, 0.5],random_state=1)
X_imb,y_imb = make_classification(n_samples=1000, weights=[0.99, 0.01],random_state=1)
def evaluate(X,y, labels):
    X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
    model = LogisticRegression(max_iter=1000).fit(X_train,y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    print(f'=== {labels} ===')
    print(f'Accuracy: {acc:.2%} ← {'MISLEADING!' if f1 < 0.5 else 'OK'}')
    print(f'Accuracy: {acc:.2%} <- {'MISLEADING!' if f1 < 0.5 else 'OK'}')
    print(f'F1 Score : {f1:.2%} ← Real performance')
    print(f'F1 Score : {f1:.2%} <- Real performance')
    print(classification_report(y_test, y_pred))
evaluate(X_bal, y_bal,'balanced data')
evaluate(X_imb, y_imb,'Imbalanced data')
X_train,X_test,y_train,y_test = train_test_split(X_imb,y_imb,test_size=0.2,random_state=42)
model_fixed = LogisticRegression(max_iter=1000).fit(X_train,y_train)
y_pred_fixed = model_fixed.predict(X_test)
print('=== After Class_Weight=balanced ===')
print(classification_report(y_test, y_pred_fixed, zero_division=0))

```

```

=== balanced data ===
Accuracy: 80.50% ← OK
Accuracy: 80.50% <- OK
F1 Score : 80.00% ← Real performance
F1 Score : 80.00% <- Real performance

```

	precision	recall	f1-score	support
0	0.84	0.78	0.81	106
1	0.77	0.83	0.80	94
accuracy			0.81	200
macro avg	0.81	0.81	0.80	200
weighted avg	0.81	0.81	0.81	200

```

=== Imbalanced data ===
Accuracy: 97.50% ← MISLEADING!
Accuracy: 97.50% <- MISLEADING!
F1 Score : 0.00% ← Real performance
F1 Score : 0.00% <- Real performance

```

	precision	recall	f1-score	support
0	0.98	0.99	0.99	196
1	0.00	0.00	0.00	4
accuracy			0.97	200
macro avg	0.49	0.50	0.49	200
weighted avg	0.96	0.97	0.97	200

```

=== After Class_Weight=balanced ===

```

	precision	recall	f1-score	support
0	0.98	0.99	0.99	196
1	0.00	0.00	0.00	4
accuracy			0.97	200
macro avg	0.49	0.50	0.49	200
weighted avg	0.96	0.97	0.97	200

```

from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
X, y = make_classification(n_samples=80000, n_features=10, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

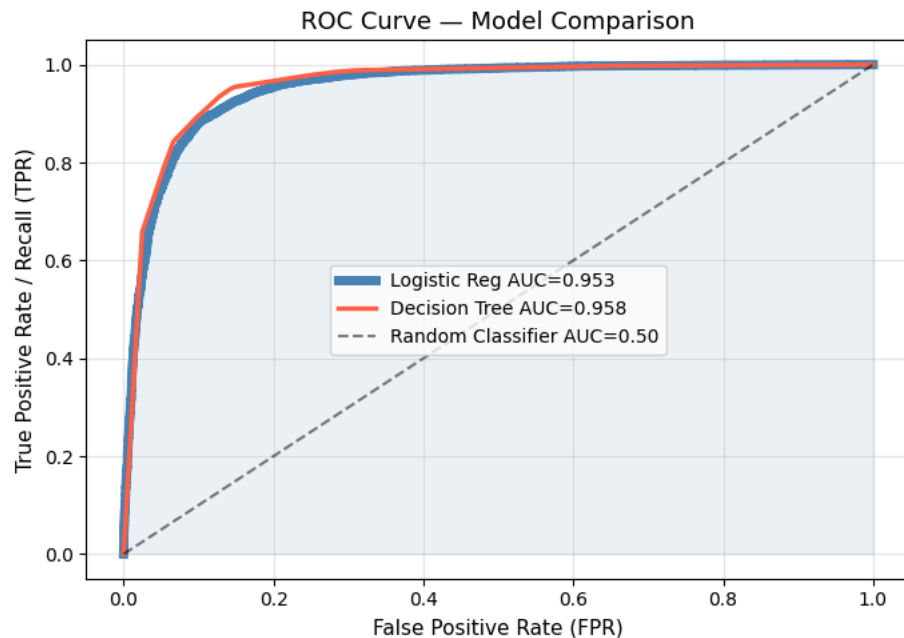
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Model 1: Logistic Regression
lr = LogisticRegression().fit(X_train, y_train)
prob_lr = lr.predict_proba(X_test)[:, 1]
# Model 2: Decision Tree (for comparison)
dt = DecisionTreeClassifier(max_depth=4).fit(X_train, y_train)
prob_dt = dt.predict_proba(X_test)[:, 1]
# Compute ROC curves

```

```

fpr_lr, tpr_lr, _ = roc_curve(y_test, prob_lr)
fpr_dt, tpr_dt, _ = roc_curve(y_test, prob_dt)
auc_lr = roc_auc_score(y_test, prob_lr)
auc_dt = roc_auc_score(y_test, prob_dt)
# Plot both on same chart
plt.figure(figsize=(7, 5))
plt.plot(fpr_lr, tpr_lr, lw=5, color='steelblue', label=f'Logistic Reg AUC={auc_lr:.3f}')
plt.plot(fpr_dt, tpr_dt, lw=2.5, color='tomato', label=f'Decision Tree AUC={auc_dt:.3f}')
plt.plot([0,1],[0,1], 'k--', alpha=0.5, label='Random Classifier AUC=0.50')
plt.fill_between(fpr_lr, tpr_lr, alpha=0.1, color='steelblue')
plt.xlabel('False Positive Rate (FPR)', fontsize=11)
plt.ylabel('True Positive Rate / Recall (TPR)', fontsize=11)
plt.title('ROC Curve - Model Comparison', fontsize=13)
plt.legend(fontsize=10); plt.grid(alpha=0.3)
plt.tight_layout(); plt.show()

```



```

print(f'Logistic Regression AUC: {auc_lr:.4f}')
print(f'Decision Tree AUC: {auc_dt:.4f}')
print('Higher AUC = better model at separating classes')

```

```

Logistic Regression AUC: 0.9833
Decision Tree AUC: 0.9761
Higher AUC = better model at separating classes

```

```

# ■ Load and Explore ■
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
ConfusionMatrixDisplay, roc_auc_score, roc_curve)

# Download the dataset if not already present
!test -f diabetes.csv || wget https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv

df = pd.read_csv('diabetes.csv') # https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
print(df.shape) # (768, 9)
print(df['Outcome'].value_counts()) # 500 healthy, 268 diabetic
print(df.describe().round(1)) # stats: mean/std/min/max
# Distribution of each feature vs outcome
df.hist(bins=25, figsize=(14,10), color='steelblue', edgecolor='white')
plt.suptitle('Feature Distributions', fontsize=14); plt.tight_layout(); plt.show()

```

(768, 9)

Outcome

0 500

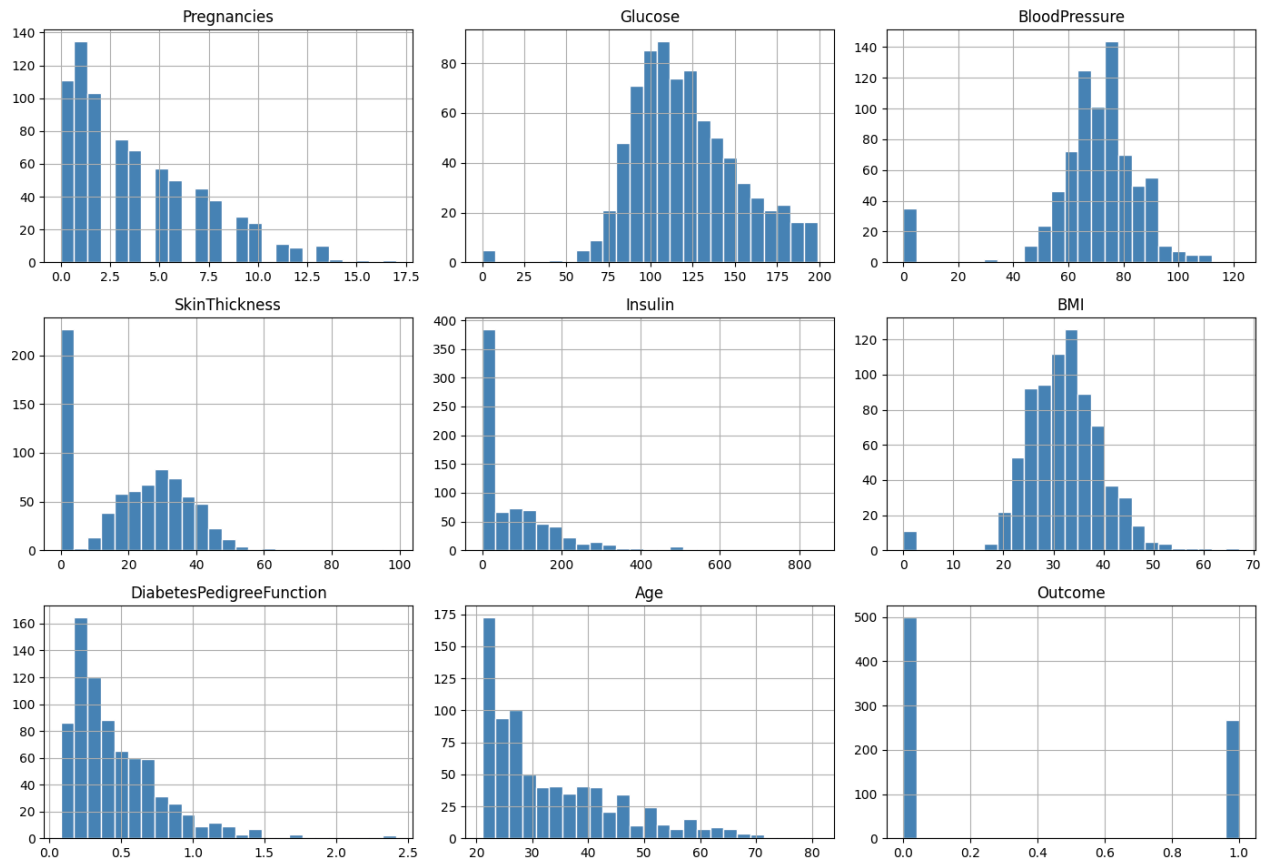
1 268

Name: count, dtype: int64

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
count	768.0	768.0	768.0	768.0	768.0	768.0	
mean	3.8	120.9	69.1	20.5	79.8	32.0	
std	3.4	32.0	19.4	16.0	115.2	7.9	
min	0.0	0.0	0.0	0.0	0.0	0.0	
25%	1.0	99.0	62.0	0.0	0.0	27.3	
50%	3.0	117.0	72.0	23.0	30.5	32.0	
75%	6.0	140.2	80.0	32.0	127.2	36.6	
max	17.0	199.0	122.0	99.0	846.0	67.1	

	DiabetesPedigreeFunction	Age	Outcome
count	768.0	768.0	768.0
mean	0.5	33.2	0.3
std	0.3	11.8	0.5
min	0.1	21.0	0.0
25%	0.2	24.0	0.0
50%	0.4	29.0	0.0
75%	0.6	41.0	1.0
max	2.4	81.0	1.0

Feature Distributions



```
# ■ Handle Missing Values ■
# Glucose, BMI, etc. cannot be 0 – these are missing values disguised as zeros
fix_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in fix_cols:
    n = (df[col] == 0).sum()
```

```
df[col] = df[col].replace(0, df[col].median())
print(f'{col:25s}: {n} zeros → replaced with median {df[col].median():.1f}')
```

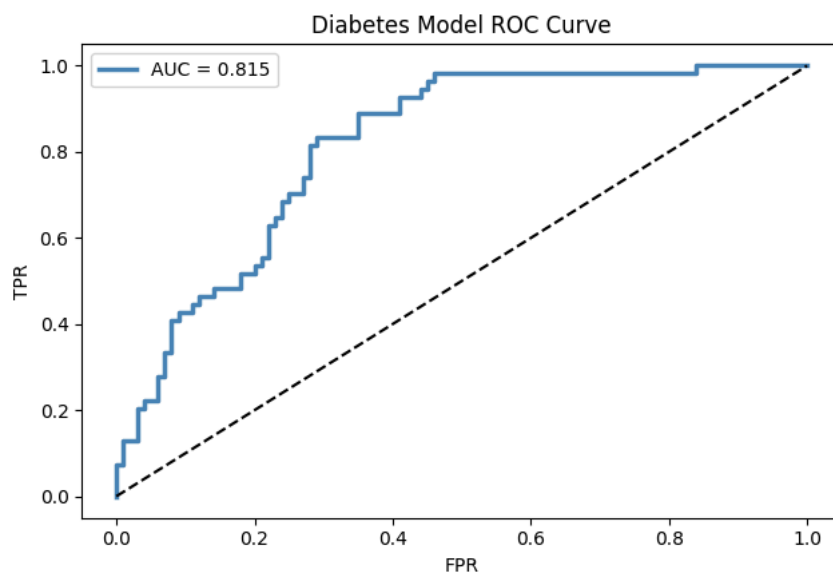
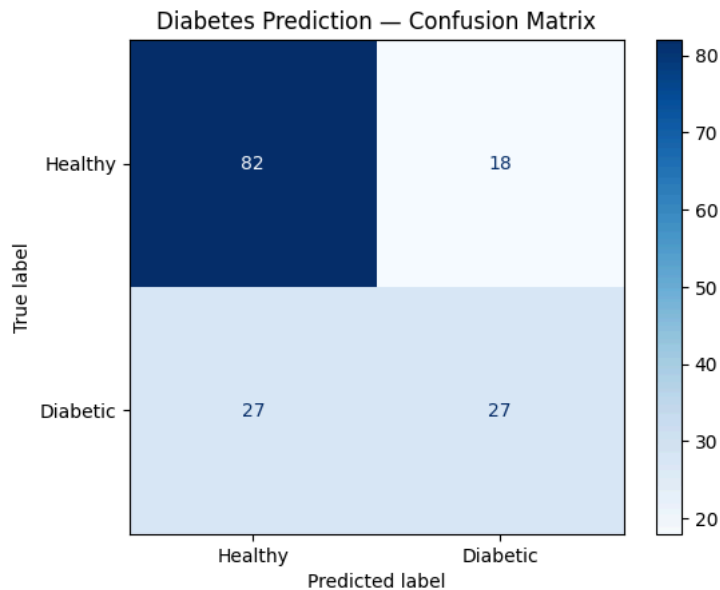
```
Glucose      : 5 zeros → replaced with median 117.0
BloodPressure : 35 zeros → replaced with median 72.0
SkinThickness : 227 zeros → replaced with median 23.0
Insulin      : 374 zeros → replaced with median 31.2
BMI          : 11 zeros → replaced with median 32.0
```

```
# ■ Split, Scale, Train ■
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y # stratify keeps class ratio
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit on train, transform train
X_test = scaler.transform(X_test) # only transform test (no leakage!)
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
# Print learned coefficient for each feature
for name, coef in zip(X.columns, model.coef_[0]):
    print(f'{name:30s}: {coef:+.3f}') # positive → increases diabetes risk
```

```
Pregnancies      : +0.376
Glucose           : +1.209
BloodPressure     : -0.054
SkinThickness     : +0.029
Insulin           : -0.133
BMI               : +0.701
DiabetesPedigreeFunction : +0.245
Age               : +0.138
```

```
# ■ Evaluate ■
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred, target_names=['Healthy', 'Diabetic']))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=['Healthy', 'Diabetic']).plot(cmap='Blues')
plt.title('Diabetes Prediction – Confusion Matrix'); plt.show()
# ROC + AUC
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(6.5, 4.5))
plt.plot(fpr, tpr, lw=2.5, color='steelblue', label=f'AUC = {auc:.3f}')
plt.plot([0,1],[0,1], 'k--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.title('Diabetes Model ROC Curve'); plt.legend(); plt.tight_layout(); plt.show()
# Expected: Accuracy ~78%, AUC ~0.84
```

	precision	recall	f1-score	support
Healthy	0.75	0.82	0.78	100
Diabetic	0.60	0.50	0.55	54
accuracy			0.71	154
macro avg	0.68	0.66	0.67	154
weighted avg	0.70	0.71	0.70	154



```
# ■ Predict New Patient + Coefficient Plot ■
# Row: [Pregnancies, Glucose, BP, SkinThickness, Insulin, BMI, DPF, Age]
new_patient = np.array([[6, 148, 72, 35, 0, 33.6, 0.627, 50]])
new_scaled = scaler.transform(new_patient) # must use same scaler
pred = model.predict(new_scaled)[0]
prob = model.predict_proba(new_scaled)[0, 1]
print(f'Prediction : {'Diabetic' if pred==1 else 'Healthy'})
print(f'Probability : {prob:.1%} chance of having diabetes')
# BONUS: Plot coefficients as a horizontal bar chart
import pandas as pd
coef_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_[0]})
coef_df = coef_df.sort_values('Coefficient')
coef_df.plot(kind='barh', x='Feature', y='Coefficient',
color=[ 'tomato' if c > 0 else 'steelblue' for c in coef_df['Coefficient']],
legend=False, figsize=(7, 4))
plt.axvline(x=0, color='black', linewidth=0.8)
plt.title('Feature Coefficients - Positive = increases diabetes risk')
plt.tight_layout(); plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
warnings.warn(
Prediction : Diabetic
Probability : 73.4% chance of having diabetes
```

